

## DECOMPRESSION SOURCE CODE

```
/*++  
  
Copyright (c) 2001-2002 Intel Corporation  
  
Module Name:  
  
    Decompress.c  
  
Abstract:  
  
    Decompressor.  
  
--*/  
  
#include "EfiCommon.h"  
  
#define BITBUFSIZ      16  
#define WNDBIT         13  
#define WNSIZ          (1U << WNDBIT)  
#define MAXMATCH       256  
#define THRESHOLD      3  
#define CODE_BIT       16  
#define UINT8_MAX      0xff  
#define BAD_TABLE      -1  
  
//  
// C: Char&Len Set; P: Position Set; T: exTra Set  
//  
  
#define NC              (0xff + MAXMATCH + 2 - THRESHOLD)  
#define CBIT            9  
#define NP              (WNDBIT + 1)  
#define NT              (CODE_BIT + 3)  
#define PBIT           4  
#define TBIT           5  
#if NT > NP  
    #define NPT         NT  
#else  
    #define NPT         NP  
#endif
```

(continues on next page)

(continued from previous page)

```
typedef struct {
    UINT8    *mSrcBase; //Starting address of compressed data
    UINT8    *mDstBase; //Starting address of decompressed data

    UINT16    mBytesRemain;
    UINT16    mBitCount;
    UINT16    mBitBuf;
    UINT16    mSubBitBuf;
    UINT16    mBufSiz;
    UINT16    mBlockSize;
    UINT32    mDataIdx;
    UINT32    mCompSize;
    UINT32    mOrigSize;
    UINT32    mOutBuf;
    UINT32    mInBuf;

    UINT16    mBadTableFlag;

    UINT8    mBuffer[WNDSIZ];
    UINT16    mLeft[2 * NC - 1];
    UINT16    mRight[2 * NC - 1];
    UINT32    mBuf;
    UINT8    mCLen[NC];
    UINT8    mPTLen[NPT];
    UINT16    mCTable[4096];
    UINT16    mPTTable[256];
} SCRATCH_DATA;

//
// Function Prototypes
//

STATIC
VOID
FillBuf (
    IN SCRATCH_DATA    *Sd,
    IN UINT16          NumOfBits
);

STATIC
VOID
Decode (
    SCRATCH_DATA    *Sd,
    UINT16          NumOfBytes
);

//
// Functions
//

EFI_STATUS
```

(continues on next page)

(continued from previous page)

```
EFIAPI
GetInfo (
    IN EFI_DECOMPRESS_PROTOCOL *This,
    IN VOID *Source,
    IN UINT32 SrcSize,
    OUT UINT32 *DstSize,
    OUT UINT32 *ScratchSize
)
/*++
```

Routine Description:

The implementation of EFI\_DECOMPRESS\_PROTOCOL.GetInfo().

Arguments:

This - Protocol instance pointer.  
 Source - The source buffer containing the compressed data.  
 SrcSize - The size of source buffer  
 DstSize - The size of destination buffer.  
 ScratchSize - The size of scratch buffer.

Returns:

EFI\_SUCCESS - The size of destination buffer and the size of scratch buffer are successful retrieved.  
 EFI\_INVALID\_PARAMETER - The source data is corrupted

```
--*/
{
    UINT8 *Src;

    *ScratchSize = sizeof (SCRATCH_DATA);

    Src = Source;
    if (SrcSize < 8) {
        return EFI_INVALID_PARAMETER;
    }

    *DstSize = Src[4] + (Src[5] << 8) + (Src[6] << 16) + (Src[7] << 24);
    return EFI_SUCCESS;
}
```

```
EFI_STATUS
EFIAPI
Decompress (
    IN EFI_DECOMPRESS_PROTOCOL *This,
    IN VOID *Source,
    IN UINT32 SrcSize,
    IN OUT VOID *Destination,
    IN UINT32 DstSize,
    IN OUT VOID *Scratch,
```

(continues on next page)

(continued from previous page)

```

    IN UINT32 ScratchSize
)
/*++

```

#### Routine Description:

The implementation of `EFI_DECOMPRESS_PROTOCOL.Decompress()`.

#### Arguments:

This - The protocol instance.  
 Source - The source buffer containing the compressed data.  
 SrcSize - The size of the source buffer  
 Destination - The destination buffer to store the decompressed data  
 DstSize - The size of the destination buffer.  
 Scratch - The buffer used internally by the decompress routine. This buffer is needed to store intermediate data.  
 ScratchSize - The size of scratch buffer.

#### Returns:

EFI\_SUCCESS - Decompression is successful  
 EFI\_INVALID\_PARAMETER - The source data is corrupted

```

--*/
{
    UINT32      Index;
    UINT16      Count;
    UINT32      CompSize;
    UINT32      OrigSize;
    UINT8       *Dst1;
    EFI_STATUS  Status;
    SCRATCH_DATA *Sd;
    UINT8       *Src;
    UINT8       *Dst;

    Status = EFI_SUCCESS;
    Src = Source;
    Dst = Destination;
    Dst1 = Dst;

    if (ScratchSize < sizeof (SCRATCH_DATA)) {
        return EFI_INVALID_PARAMETER;
    }

    Sd = (SCRATCH_DATA *)Scratch;

    if (SrcSize < 8) {
        return EFI_INVALID_PARAMETER;
    }

    CompSize = Src[0] + (Src[1] << 8) + (Src[2] << 16) + (Src[3] << 24);

```

(continues on next page)

(continued from previous page)

```

OrigSize = Src[4] + (Src[5] << 8) + (Src[6] << 16) + (Src[7] << 24);

if (SrcSize < CompSize + 8) {
    return EFI_INVALID_PARAMETER;
}

Src = Src + 8;

for (Index = 0; Index < sizeof(SCRATCH_DATA); Index++) {
    ((UINT8*)Sd)[Index] = 0;
}

Sd->mBytesRemain = (UINT16)(-1);
Sd->mSrcBase = Src;
Sd->mDstBase = Dst;
Sd->mCompSize = CompSize;
Sd->mOrigSize = OrigSize;

//
// Fill the first two bytes
//
FillBuf(Sd, BITBUFSIZ);

while (Sd->mOrigSize > 0) {

    Count = (UINT16) (WNDSIZ < Sd->mOrigSize? WNDSIZ: Sd->mOrigSize);
    Decode (Sd, Count);

    if (Sd->mBadTableFlag != 0) {
        //
        // Something wrong with the source
        //
        return EFI_INVALID_PARAMETER;
    }

    for (Index = 0; Index < Count; Index ++) {
        if (Dst1 < Dst + DstSize) {
            *Dst1++ = Sd->mBuffer[Index];
        } else {
            return EFI_INVALID_PARAMETER;
        }
    }

    Sd->mOrigSize -= Count;
}

if (Sd->mBadTableFlag != 0) {
    Status = EFI_INVALID_PARAMETER;
} else {
    Status = EFI_SUCCESS;
}

```

(continues on next page)

(continued from previous page)

```

    return Status;
}

STATIC
VOID
FillBuf (
    IN SCRATCH_DATA *Sd,
    IN UINT16 NumOfBits
)
/*++

Routine Description:

    Shift mBitBuf NumOfBits left. Read in NumOfBits of bits from source.

Arguments:

    Sd - The global scratch data
    NumOfBit - The number of bits to shift and read.

Returns: (VOID)

--*/
{
    Sd->mBitBuf = (UINT16)(Sd->mBitBuf << NumOfBits);

    while (NumOfBits > Sd->mBitCount) {

        Sd->mBitBuf |= (UINT16)(Sd->mSubBitBuf <<
            (NumOfBits = (UINT16)(NumOfBits - Sd->mBitCount)));

        if (Sd->mCompSize > 0) {

            //
            // Get 1 byte into SubBitBuf
            //
            Sd->mCompSize--;
            Sd->mSubBitBuf = 0;
            Sd->mSubBitBuf = Sd->mSrcBase[Sd->mInBuf++];
            Sd->mBitCount = 8;

        } else {

            Sd->mSubBitBuf = 0;
            Sd->mBitCount = 8;

        }

    }

    Sd->mBitCount = (UINT16)(Sd->mBitCount - NumOfBits);
    Sd->mBitBuf |= Sd->mSubBitBuf >> Sd->mBitCount;
}

```

(continues on next page)

(continued from previous page)

```

STATIC
UINT16
GetBits(
    IN SCRATCH_DATA *Sd,
    IN UINT16 NumOfBits
)
/*++

```

**Routine Description:**

Get NumOfBits of bits out from mBitBuf. Fill mBitBuf with subsequent NumOfBits of bits from source. Returns NumOfBits of bits that are popped out.

**Arguments:**

Sd - The global scratch data.  
 NumOfBits - The number of bits to pop and read.

**Returns:**

The bits that are popped out.

```

--*/
{
    UINT16 OutBits;

    OutBits = (UINT16)(Sd->mBitBuf >> (BITBUFSIZ - NumOfBits));

    FillBuf (Sd, NumOfBits);

    return OutBits;
}

```

```

STATIC
UINT16
MakeTable (
    IN SCRATCH_DATA *Sd,
    IN UINT16 NumOfChar,
    IN UINT8 *BitLen,
    IN UINT16 TableBits,
    OUT UINT16 *Table
)
/*++

```

**Routine Description:**

Creates Huffman Code mapping table according to code length array.

**Arguments:**

Sd - The global scratch data

(continues on next page)

(continued from previous page)

NumOfChar - Number of symbols in the symbol set  
 BitLen - Code length array  
 TableBits - The width of the mapping table  
 Table - The table

Returns:

0 - OK.  
 BAD\_TABLE - The table is corrupted.

```
--*/
{
    UINT16 Count[17];
    UINT16 Weight[17];
    UINT16 Start[18];
    UINT16 *p;
    UINT16 k;
    UINT16 i;
    UINT16 Len;
    UINT16 Char;
    UINT16 JuBits;
    UINT16 Avail;
    UINT16 NextCode;
    UINT16 Mask;
    UINT16 MaxTableLength;

    for (i = 1; i <= 16; i++) {
        Count[i] = 0;
    }

    for (i = 0; i < NumOfChar; i++) {
        if (BitLen[i] > 16) {
            return (UINT16) BAD_TABLE;
        }
        Count[BitLen[i]]++;
    }

    Start[1] = 0;

    for (i = 1; i <= 16; i++) {
        Start[i + 1] = (UINT16)(Start[i] + (Count[i] << (16 - i)));
    }

    if (Start[17] != 0) { /*(1U << 16)*/
        return (UINT16)BAD_TABLE;
    }

    JuBits = (UINT16)(16 - TableBits);

    for (i = 1; i <= TableBits; i++) {
        Start[i] >>= JuBits;
        Weight[i] = (UINT16)(1U << (TableBits - i));
    }
}
```

(continues on next page)



(continued from previous page)

```

}

while (i <= 16) {
    Weight[i++] = (UINT16)(1U << (16 - i));
}

i = (UINT16)(Start[TableBits + 1] >> JuBits);

if (i != 0) {
    k = (UINT16)(1U << TableBits);
    while (i != k) {
        Table[i++] = 0;
    }
}

Avail = NumOfChar;
Mask = (UINT16)(1U << (15 - TableBits));
MaxTableLength = (UINT16) (1U << TableBits);

for (Char = 0; Char < NumOfChar; Char++) {

    Len = BitLen[Char];
    if (Len == 0) {
        continue;
    }

    NextCode = (UINT16)(Start[Len] + Weight[Len]);
    if (Len <= TableBits) {

        for (i = Start[Len]; i < NextCode; i++) {
            if (i >= MaxTableLength) {
                return (UINT16) BAD_TABLE;
            }
            Table[i] = Char;
        }

    } else {

        k = Start[Len];
        p = &Table[k >> JuBits];
        i = (UINT16)(Len - TableBits);

        while (i != 0) {
            if (*p == 0) {
                Sd->mRight[Avail] = Sd->mLeft[Avail] = 0;
                *p = Avail++;
            }

            if (k & Mask) {
                p = &Sd->mRight[*p];
            } else {
                p = &Sd->mLeft[*p];
            }
        }
    }
}

```

(continues on next page)

(continued from previous page)

```

    }

    k <<= 1;
    i --;
}

*p = Char;

}

Start[Len] = NextCode;
}

//
// Succeeds
//
return 0;
}

STATIC
UINT16
DecodeP (
    IN SCRATCH_DATA *Sd
)
/*++

Routine description:

    Decodes a position value.

Arguments:

    Sd - the global scratch data

Returns:

    The position value decoded.

--*/
{
    UINT16 Val;
    UINT16 Mask;

    Val = Sd->mPTTable[Sd->mBitBuf >> (BITBUFSIZ - 8)];

    if (Val >= NP) {
        Mask = 1U << (BITBUFSIZ - 1 - 8);

        do {

            if (Sd->mBitBuf & Mask) {
                Val = Sd->mRight[Val];
            }
        } while (Mask > 0);
    }
}

```

(continues on next page)

(continued from previous page)

```

    } else {
        Val = Sd->mLeft[Val];
    }

    Mask >>= 1;
} while (Val >= NP);
}

//
// Advance what we have read
//
FillBuf (Sd, Sd->mPTLen[Val]);

if (Val) {
    Val = (UINT16)((1U << (Val - 1)) + GetBits (Sd, (UINT16)(Val - 1)));
}

return Val;
}

STATIC
UINT16
ReadPTLen (
    IN SCRATCH_DATA *Sd,
    IN UINT16 nn,
    IN UINT16 nbit,
    IN UINT16 Special
)
/*++

```

#### Routine Description

Reads code lengths for the Extra Set or the Position Set

#### Arguments:

Sd - The global scratch data  
nn - Number of symbols  
nbit - Number of bits needed to represent nn  
Special - The special symbol that needs to be taken care of

#### Returns:

0 - OK.  
BAD\_TABLE - Table is corrupted.

```

--*/
{
    UINT16 n;
    UINT16 c;
    UINT16 i;
    UINT16 Mask;

```

(continues on next page)

(continued from previous page)

```

n = GetBits (Sd, nbit);

if (n == 0) {
    c = GetBits (Sd, nbit);

    for ( i = 0; i < 256; i ++ ) {
        Sd->mPTTable[i] = c;
    }

    for ( i = 0; i < nn; i++) {
        Sd->mPTLen[i] = 0;
    }

    return 0;
}

i = 0;

while (i < n) {

    c = (UINT16)(Sd->mBitBuf >> (BITBUFSIZ - 3));

    if (c == 7) {
        Mask = 1U << (BITBUFSIZ - 1 - 3);
        while (Mask & Sd->mBitBuf) {
            Mask >>= 1;
            c += 1;
        }
    }

    FillBuf (Sd, (UINT16)((c < 7) ? 3 : c - 3));

    Sd->mPTLen [i++] = (UINT8)c;

    if (i == Special) {

        c = GetBits (Sd, 2);
        while ((INT16)(--c) >= 0) {
            Sd->mPTLen[i++] = 0;
        }
    }

    while (i < nn) {
        Sd->mPTLen [i++] = 0;
    }

    return ( MakeTable (Sd, nn, Sd->mPTLen, 8, Sd->mPTTable) );
}

```

STATIC

(continues on next page)

(continued from previous page)

```

VOID
ReadCLen (
    SCRATCH_DATA *Sd
)
/*++

Routine Description:

    Reads code lengths for Char&Len Set.

Arguments:

    Sd - the global scratch data

Returns: (VOID)

--*/
{
    UINT16 n;
    UINT16 c;
    UINT16 i;
    UINT16 Mask;

    n = GetBits(Sd, CBIT);

    if (n == 0) {
        c = GetBits(Sd, CBIT);

        for (i = 0; i < NC; i ++) {
            Sd->mCLen[i] = 0;
        }

        for (i = 0; i < 4096; i ++) {
            Sd->mCTable[i] = c;
        }

        return;
    }

    i = 0;
    while (i < n) {

        c = Sd->mPTTable[Sd->mBitBuf >> (BITBUFSIZ - 8)];
        if (c >= NT) {
            Mask = 1U << (BITBUFSIZ - 1 - 8);

            do {

                if (Mask & Sd->mBitBuf) {
                    c = Sd->mRight [c];
                } else {
                    c = Sd->mLeft [c];
                }
            } while (Mask > 0);
        }
        i++;
    }
}

```

(continues on next page)

(continued from previous page)

```

    }

    Mask >>= 1;

    }while (c >= NT);
}

//
// Advance what we have read
//
FillBuf (Sd, Sd->mPTLen[c]);

if (c <= 2) {

    if (c == 0) {
        c = 1;
    } else if (c == 1) {
        c = (UINT16)(GetBits (Sd, 4) + 3);
    } else if (c == 2) {
        c = (UINT16)(GetBits (Sd, CBIT) + 20);
    }

    while ((INT16)(--c) >= 0) {
        Sd->mCLen[i++] = 0;
    }

} else {

    Sd->mCLen[i++] = (UINT8)(c - 2);

}

while (i < NC) {
    Sd->mCLen[i++] = 0;
}

MakeTable (Sd, NC, Sd->mCLen, 12, Sd->mCTable);

return;
}

STATIC
UINT16
DecodeC (
    SCRATCH_DATA *Sd
)
/*++

```

**Routine Description:**

Decode a character/length value.

(continues on next page)

(continued from previous page)

Arguments:

Sd - The global scratch data.

Returns:

The value decoded.

```
--*/
{
    UINT16 j;
    UINT16 Mask;

    if (Sd->mBlockSize == 0) {

        //
        // Starting a new block
        //

        Sd->mBlockSize = GetBits(Sd, 16);
        Sd->mBadTableFlag = ReadPTLen (Sd, NT, TBIT, 3);
        if (Sd->mBadTableFlag != 0) {
            return 0;
        }

        ReadCLen (Sd);

        Sd->mBadTableFlag = ReadPTLen (Sd, NP, PBIT, (UINT16)(-1));
        if (Sd->mBadTableFlag != 0) {
            return 0;
        }
    }

    Sd->mBlockSize --;
    j = Sd->mCTable[Sd->mBitBuf >> (BITBUFSIZ - 12)];

    if (j >= NC) {
        Mask = 1U << (BITBUFSIZ - 1 - 12);

        do {
            if (Sd->mBitBuf & Mask) {
                j = Sd->mRight[j];
            } else {
                j = Sd->mLeft[j];
            }

            Mask >>= 1;
        } while (j >= NC);
    }

    //

```

(continues on next page)

(continued from previous page)

```
// Advance what we have read
//
FillBuf(Sd, Sd->mClen[j]);

return j;
}
```

```
STATIC
VOID
Decode (
    SCRATCH_DATA *Sd,
    UINT16 NumOfBytes
)
/*++
```

Routine Description:

Decode NumOfBytes and put the resulting data at starting point of mBuffer.  
The buffer is circular.

Arguments:

Sd - The global scratch data  
NumOfBytes - Number of bytes to decode

Returns: (VOID)

```
--*/
{
    UINT16 di;
    UINT16 r;
    UINT16 c;

    r = 0;
    di = 0;

    Sd->mBytesRemain --;
    while ((INT16)(Sd->mBytesRemain) >= 0) {
        Sd->mBuffer[di++] = Sd->mBuffer[Sd->mDataIdx++];

        if (Sd->mDataIdx >= WNDSIZ) {
            Sd->mDataIdx -= WNDSIZ;
        }

        r ++;
        if (r >= NumOfBytes) {
            return;
        }
        Sd->mBytesRemain --;
    }

    for (;;) {
```

(continues on next page)



(continued from previous page)

```

c = DecodeC (Sd);
if (Sd->mBadTableFlag != 0) {
    return;
}

if (c < 256) {

    //
    // Process an Original character
    //

    Sd->mBuffer[di++] = (UINT8)c;
    r ++;
    if (di >= WNDSIZ) {
        return;
    }

} else {

    //
    // Process a Pointer
    //

    c = (UINT16)(c - (UINT8_MAX + 1 - THRESHOLD));
    Sd->mBytesRemain = c;

    Sd->mDataIdx = (r - DecodeP(Sd) - 1) & (WNDSIZ - 1); //Make circular

    di = r;

    Sd->mBytesRemain --;
    while ((INT16)(Sd->mBytesRemain) >= 0) {
        Sd->mBuffer[di++] = Sd->mBuffer[Sd->mDataIdx++];
        if (Sd->mDataIdx >= WNDSIZ) {
            Sd->mDataIdx -= WNDSIZ;
        }

        r ++;
        if (di >= WNDSIZ) {
            return;
        }

        if (di >= Sd->mOrigSize) {
            Sd->mBadTableFlag = (UINT16) BAD_TABLE;
            return;
        }

        Sd->mBuffer[di++] = Sd->mBuffer[Sd->mDataIdx++];
        Sd->mBytesRemain --;
    }
}

```

(continues on next page)

(continued from previous page)

```
return;  
}
```